CSSE2010/7201

Assignment (AVR Project)

Semester One, 2026

Due: 4:00pm, Friday 22nd May

Weighting: 10%

The University of Queensland

School of Electrical Engineering and Computer Science

Objective

As part of the assessment for this course, you are required to undertake an AVR project which will test you against some of the more practical learning objectives of the course, namely your C programming skills applied to the ATmega324A microcontroller.

You are required to construct a program to implement a morse code emulator (described in detail on page 5). The AVR ATmega324A microcontroller will run the program and receive input from multiple sources, and use an LED matrix as a display, with additional information outputted to a serial terminal, the seven-segment display and other devices.

The version of the morse code emulator provided to you has very basic functionality. You will be adding features such as inputting dots and dashes, displaying the encoded characters on the LED matrix, and interacting with serial communication. The various features have different tiers of difficulty and will each contribute a certain number of marks. Note that marks are awarded based on demonstrated functionality only, regardless of how much effort or code has gone into attempting such functionality.

Don't Panic!

You have been provided with approximately 900 lines of code to start with – many of which are comments. Whilst this code may seem confusing, you do not need to understand all of it initially. To start with, you should read the header (`.h`) files provided, along with all of `morse.c`. You may need to look at the AVR C Library documentation to understand some of the functions used. Lecture 19 gives a demonstration of some of the expected functionality to be implemented and walks through setting the project up with the provided base code, and how to submit. Note that the requirements in this document takes priority over anything shown in the lecture.

Grading Note

As described in the course profile, if you do not obtain at least 50% on this AVR assignment (before any late penalty), then your course grade will be capped at a 5. Your AVR assignment mark (after any late penalty) will count 10% towards your final course grade.

In addition, features are grouped into tiers, and the features that comprise each tier have more marks assigned than the total marks allocated for that tier. You cannot earn more marks than the total allocation, however, you will be given all marks you are awarded for each feature up to this limit. This means that you can still get full marks for a tier, even if you are not awarded full marks for each feature, if you complete features with marks that would total above the allocated marks for that tier.

The marks allocated and available for each tier are:

- Tier A – 40 marks allocated with 44 marks assigned to its features;
- Tier B – 30 marks allocated with 34 marks assigned to its features;
- Tier C – 20 marks allocated with 24 marks assigned to its features;
- Demonstration – 10 marks allocated, plus overall mark cap.

Your total marks for Tier B cannot exceed, and will be capped by, your total marks for Tier A. Your total marks for Tier C cannot exceed, and will be capped by, your total marks for Tier B.

Code Style

No marks are awarded nor deducted in the marking criteria for code style. However, poorly styled code will be detrimental to debugging, expanding and demonstrating the code, and so may lead to a loss of marks that way. If your code is sufficiently poorly styled, then asking for assistance from course staff may be refused until your style is improved. This includes, but is not limited to, if your code:

- Utilises `goto` or digraphs/trigraphs;
- Fails to bitshift where doing so would be expected;
- Declares variables without meaningful names;
- Uses integer variables without exact width declarations (e.g. `int` instead of `int8_t`);
- Has inconsistent or contradictory indentation and/or brace positioning.

Demonstration

As part of this assessment task, you must demonstrate the functionality of your implemented system. You will be assigned a 15–25 minute block during one of your two regularly scheduled practical sessions in week 13. Your exact demo time slot will be communicated to you on BlackBoard, with a timeslot and an associated mnemonic codeword – this codeword is one-to-one with your timeslot, but has no additional meaning. You are expected to arrive on time, enter the room, and await instructions from the teaching staff. Before your demo is to begin, you will have 5 minutes to power up, wire up, and quickly test your system.

During the demonstration, we will compile the code you submitted on BlackBoard using Microchip Studio 7 and program it onto your microcontroller with the Axon 47-104 laboratory computers. As there is no time during the demo for debugging, you must check that your code compiles under these conditions before submission. Your submission will be marked according

to the functionality demonstrated on the lab computers. Any discrepancy between the behaviour on this computer and any other computer will not be considered a justification for a remark.

Once the system has been programmed, you will be asked to demonstrate its functionality. To evaluate both your understanding of the implementation, as well as robustness of your design, you will be asked questions about your code, which may include being asked to make minor adjustments to tweak certain features, or how you would go about modifying your code for larger changes. As such, you will be expected to be able to navigate your code efficiently – a good file structure and code comments may be beneficial in this regard.

Should you fail to attend your allocated demonstration time during week 13, you will be offered an opportunity to attend a make-up demonstration in the first week of the exam period. There can be no additional demo sessions beyond this period, to allow grade finalisation. If you do not have a valid extension for this assessment, your mark will be capped at 50%. If you have an approved extension for the Blackboard submission, then you will be communicated a demo time after the extended due date for the submission. If you do not attend either demo session, you will receive 0 for this assignment, regardless of any code functionality.

Additionally, should you arrive late to your allocated demo slot, you will incur a 10% penalty to your assignment grade for every five minutes, or part thereof, that you are late.

Demonstration & Grading (10 Marks + Cap)

You will be given a mark out of 10 for your understanding during your demonstration, dependent on how well you are able to answer the marker's questions about your code, methods, and so forth. This will be added to your assignment mark as well as act as a cap, according to this table:

Understanding demo mark	Maximum assignment mark
No Demo	0%
0/10	20%
1/10	30%
2/10	40%
3/10	50%
4/10	60%

Understanding demo mark	Maximum assignment mark
5/10	70%
6/10	80%
7/10	85%
8/10	90%
9/10	95%
10/10	100%

Academic Merit, Plagiarism, Collusion and Other Misconduct

You should read and understand the statement on academic merit, plagiarism, collusion and other misconduct contained within the course profile and the document referenced in that course profile. You must not show your code to, or share your code with, any other student under any circumstances. You must not post your code to public discussion forums or save your code in publicly accessible repositories. You must not look at or copy code from any other student. All submitted files will be subject to electronic plagiarism detection and misconduct proceedings will be instituted against students where plagiarism or collusion is suspected. The electronic plagiarism detection can detect similarities in code structure even if comments, variable names, formatting etc. are modified.

Additionally, while AI can help in learning, it should not be relied upon for feature implementation in this assignment. In addition to the usual caveats about AI, it will only give general answers unless given detailed prompts, and so will not be able to give correct answers to some specific aspects of the components used in this course. Furthermore, an overreliance on AI without student learning will lead to poor performance during the demos.

Program Description

The program you will be provided with has several C files which contain groups of related functions. The files provided are described below. The corresponding `.h` files (except for `morse.c`) list the functions that are intended to be accessible from other files. You may modify any of the provided files. You must submit all files used to build your project, even if you have not modified some provided files. Many files make assumptions about which AVR ports are used to connect to various IO devices. You are encouraged not to change these.

- `morse.c` – this is the main file that contains the splash screen code, as well as calling the setup functions for the other files.
- `encoding.c/.h` – the function in this file will, if given a Morse signal value, return the character represented by that value.
- `display.c/.h` – here you will find the pattern for the splash screen illustration, as well as converting from a character to its LED matrix glyph. You are likely to want to complete `draw_small_char` early in the assignment.
- `ledmatrix.c/.h` – this contains functions which give easier access to the services provided by the LED matrix. It makes use of the SPI routines implemented in `spi.c`. You are encouraged to add functions for the other commands possible with the EECS LED matrix. Semantically, this handles LED matrix related functionality that is generic to any program, while `display.c/.h` handles functionality related specifically to the Morse code emulator.
- `serialio.c/.h` – this file is responsible for handling serial input and output using interrupts. It also maps the C standard IO routines (e.g., `printf()` and `fgetc()`) to use the serial interface so you are able to use `printf()` etc. for debugging purposes if you wish. You should not need to look in this file, but you may be interested in how it works,

and the buffer sizes used for input and output (and what happens when the buffers fill up).

- `terminalio.c/.h` – this encapsulates the sending of various escape sequences which enable some control over terminal appearance and text placement – you can call these functions (declared in `terminalio.h`) instead of remembering various escape sequences. Additional information about terminal IO will be provided on the course BlackBoard site. You are unlikely to need to modify this file until you reach tier C.
- `spi.c/.h` – this file encapsulates all SPI communication. Note that by default, all SPI communication uses busy waiting (i.e., polling) – the "send" routine returns only when the data is sent. If you need the CPU cycles for other activities, you may wish to consider converting this to interrupt based IO, similar to the way that serial IO is handled.

NB: When you create a new project in Microchip Studio, a `main.c` file will automatically be created, containing an empty `main()` function. `morse.c` also contains a `main()` function, but Microchip Studio will preferentially look for the `main()` function in the `main.c` file, if it exists. Please ensure that you delete the `main.c` file so that Microchip Studio will look for the `main()` function in `morse.c`.

You are encouraged to make new files as desired in a similar manner to these initial files. For example, adding `timer0.c/.h` for the functionality that depends on timer/counter 0 (and similarly for timers/counters 1 and 2), `joystick.c/.h` or `adc.c/.h` for the joystick functionality, etc.

Morse Code Description

Morse code is a system of encoding messages with a binary (high/low) signal. A message has a given "beat" duration, which may vary depending on the operator – for this assignment, it will eventually be around 100ms.

- If the signal is held high for one beat, that is a "dot" mark (represented as "●";
- If the signal is held high for three beats, that is a "dash" mark (represented as "▬");
- If the signal is held low for one beat, that is a gap between marks;
- If the signal is held low for three beats, that is a gap between characters;
- If a signal is held low for five beats, that is a gap between words (this has been shortened for this assignment from the standard seven beats).

Historically, one of the most common ways that Morse code was used was in the telegraph service. Here, a high signal would produce a tone, and a low signal would produce silence.

Different combinations of marks encode different characters; see Figure 1.

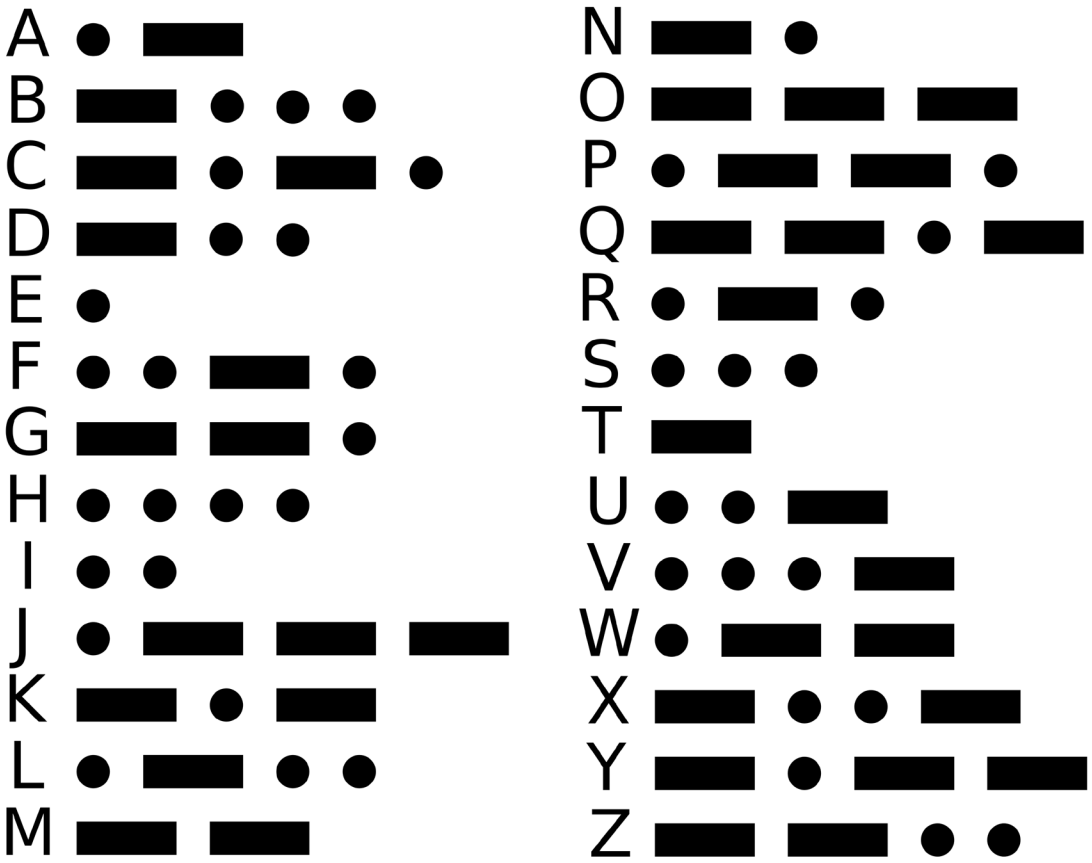


Figure 1: Morse code letter encodings

For example, Figure 2 shows how the message "CSSE" would be encoded.

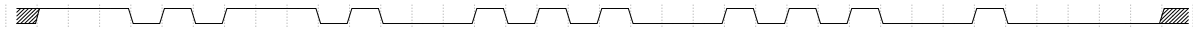


Figure 2: Timing diagram for Morse encoding of "CSSE"

Initial Operation

The provided program has very limited functionality. It will display a splash screen which detects the rising edge on the push buttons B0, B1, or B2. This button press will exit the splash screen, and blank the LED matrix.

Wiring Advice

When completing this AVR assignment, you will need to make additional connections to the ATmega324A microcontroller to implement particular features. To do this, you will need to choose which pins to make these connections to. There are multiple ways to do this, so the exact wiring configuration will be left up to you. Hint: Before implementing any features, read through them all, and consider what peripherals each feature requires and any pin limitations this imposes. If you do not do this, you may find yourself in a situation where the pins that must be used for a peripheral for a later feature are already connected to another peripheral, requiring you to rewire your breadboard and update your code before you can use the new peripheral. Some connections are defined for you in the provided base code and are shown in grey in this table.

Of the 32 pins available on the ATmega324A, a fully featured assignment will use 31 of them.

Additionally, you may wish to plan out your use of the three timers/counters available before implementing any features, for similar reasons.

Wiring Table

Port	Pin 7	Pin 6	Pin 5	Pin 4	Pin 3	Pin 2	Pin 1	Pin 0
A								
B	SPI connection to LED matrix					Button B2	Button B1	Button B0
C								
D							Serial RX	Serial TX
							Baud rate: 19200	

Program Features

Marks will be awarded for features as described below. Part marks will be awarded if part of the specified functionality is demonstrated. Marks are awarded only for demonstrated functionality in the final submission – no marks are awarded for attempting to implement the functionality, no matter how much effort has gone into it, if there is no evidence of functionality when the program is run. You may implement higher-tier features without implementing all lower-tier features if you like, subject to prerequisite requirements. However, be aware that heavy penalties will apply to any feature that has been completed if its prerequisite feature(s) have not been adequately completed. The number of marks is not an indication of difficulty. It is much easier to earn the first 50% of marks than the second 50%, and marks may be deducted if features interact negatively with one another, and the number of potential iterations grows quadratically with the number of features implemented.

You may modify any of the code provided (unless indicated otherwise) and use any of the code from learning lab sessions and/or posted on the course BlackBoard site. For some of the easier features, the description below may tell you which code to modify or there may be comments in the supplied code to guide you.

Minimum Performance — Tier X: Pass/Fail

Your program must have at least the features present in the code supplied to you, i.e., it must build and run, show the splash screen, and clear the LED matrix when buttons B0, B1 or B2 is pressed. No marks can be earned for other features unless this requirement is met, i.e., your AVR assignment mark will be zero.

Button Inputs — Tier A: 8 marks

Modify the program so that, after clearing the splash screen, when the buttons on the IO board are pressed, data is inputted. Button B0 inputs a dot, button B1 inputs a dash, and button B2 inputs a "submission". The submission input should signal the end of the current character, or if pressed twice in a row without inputting a mark, signal the end of the current word. Pressing the submit button more than twice in a row should be ignored i.e. there cannot be multiple gaps between words in a row.

The history of the inputs should be shown on the IO board LEDs, with one LED for each beat. The newest inputs should appear on the right, and move to the left as additional buttons are pressed. Gaps between marks should be automatically added, but the correct spacing between characters and words (three and five beats respectively) should be maintained.

For example:

Input	LEDs
(Initial)	●●●●●●●●
B0 – Dot	●●●●●●●●
B1 – Dash	●●●●●●●●
B0 – Dot	●●●●●●●●
B1 – Dash	●●●●●●●●
B2 – Submit	●●●●●●●●
B2 – Submit	●●●●●●●●

Here, a black circle represents an off LED, while the coloured circles represent an on LED. The colour of the on LEDs do not reflect the colours of the LEDs on the IO board, but are coloured to indicate which input produced them.

At this stage, the duration that the button is pressed should have no effect; the inputs should trigger as the button is pressed, and not as it is released. As a caveat of this, if one button is pressed and held, and another button is tapped, the latter button should trigger its effect.

LED Matrix – Single Character — Tier A: 8 marks

*Requires the **Button Inputs** feature to be implemented*

When a character is submitted, it should be drawn on the LED matrix, abutting the right (high x) edge, in green. A submission without any marks inputted should draw a space i.e. an empty area. However, multiple consecutive spaces should be impossible; repeated presses of the submit button should be ignored beyond the first space. An invalid sequence of marks should display a "?".

The base code has functions for converting a sequence of marks to a character. The marks are encoded into an eight-bit integer, where a "0" represents a dot, and a "1" represents a dash. The marks are prefixed with a single "1", and everything to the left of that prefix is a "0" – this gives every character a unique representation (otherwise, for example, "E", "I", "S" and "H", which each consist solely of dots, would all have the same encoding of a string of "0"s). Some examples of this encoding scheme are shown in this table (with the non-prefix bits of the encoding underlined):

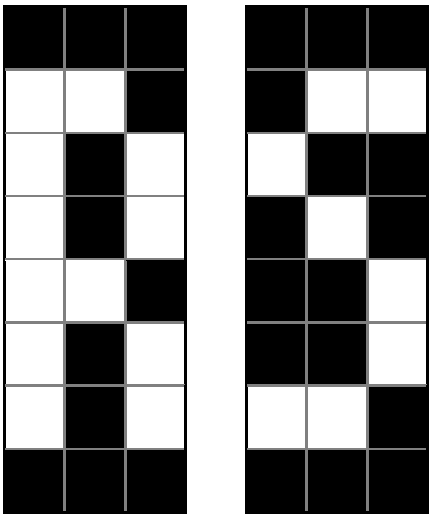
Character	Morse Encoding	Binary Encoding
[empty]		0000 0001
E	●	0000 001<u>0</u>
I	●●	0000 01<u>00</u>

Character	Morse Encoding	Binary Encoding
S	●●●	0000 1<u>000</u>
H	●●●●	0001 <u>0000</u>
T	▬	0000 001<u>1</u>

Character	Morse Encoding	Binary Encoding
M	▬ ▬	0000 0111
O	▬ ▬ ▬	0000 1111
A	●▬	0000 0101

Character	Morse Encoding	Binary Encoding
D	▬●●	0000 1100
C	▬●▬●	0001 1010
?	e.g.: ●●●▬ ▬ ▬	0100 0111

The base code also has functions for converting a character to its LED matrix glyph. Each glyph is stored as a three-element array of `uint8_t`s, with a high bit representing a lit pixel and a low bit representing a dark pixel. The first element of each array represents the left column, and the most significant bit of each element represent the pixel in the top row. For example, "R" and "S" are represented as `{ 0b01111110, 0b01001000, 0b00110110 }` and `{ 0b00100010, 0b01010010, 0b01001100 }`, respectively. This would produce the glyphs:



NB: The **Font Selection** feature below will implement the use of a five-column wide glyph for each character, for which a function is also present in the base code.

For example, inputting "▬●▬●Ω" (using "Ω" to represent the submission input) should result in the LED matrix looking like this:

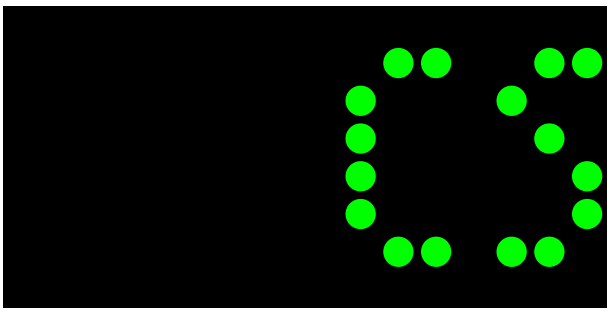
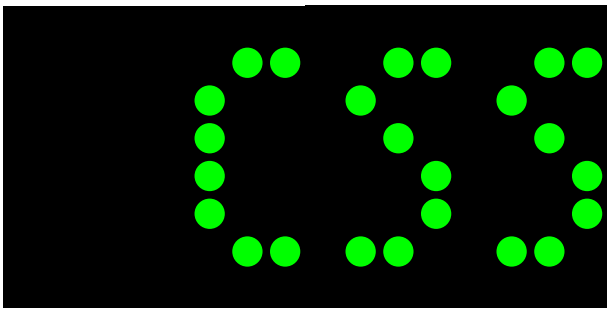
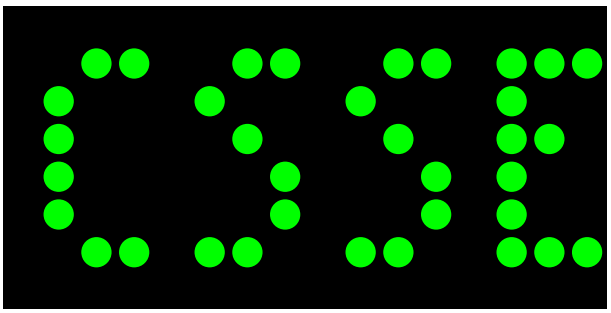
Your implementation should make efficient use of the available LED matrix commands, and avoid sending gratuitously more commands than the minimum necessary. For this feature, it can be accomplished by sending 30 SPI bytes to the LED matrix for each glyph; any implementation that uses up to 45 bytes per glyph (50% more than the lower bound) will satisfy this paragraph's requirement. You will be asked to justify during your demo that the number of commands sent

is appropriate. You need not implement additional LED matrix commands than those included in the base code at this stage.

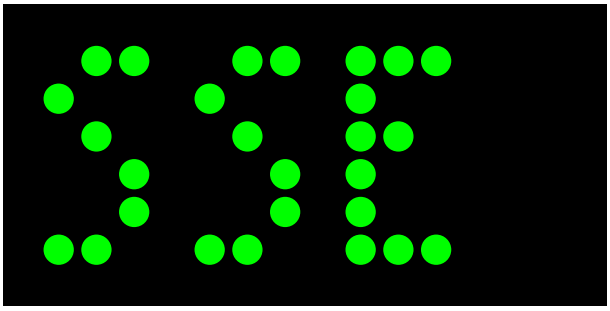
LED Matrix – Multiple Characters — Tier A: 6 marks

*Requires the **LED Matrix – Single Character** feature to be implemented.*

When additional characters are inputted, the previous character(s) should first move four columns to the left, so that they are adjacent to the new character. For example:

Input	LED Matrix
"▬●▬●Ω"	
"●●●Ω"	
"●●●Ω"	
"●Ω"	

"Ω"



After four characters have been submitted, any additional submissions will cause the oldest (leftmost) character to be removed from the LED matrix.

Your implementation should make efficient use of the available LED matrix commands, and avoid sending gratuitously more commands than the minimum necessary. You will be asked to justify during your demo that the number of commands sent is appropriate. You will now need to implement additional LED matrix commands beyond those included in the base code to do this feature efficiently.

NB: If **Animation – LED Matrix** has been implemented, then the newly submitted character should once again appear abutting the right edge, then immediately animate moving left without user input. Otherwise, if **LED Matrix – Incomplete Character** has been implemented, a newly submitted character should immediately move left when submitted, to make room for the next incomplete character. Otherwise, if neither of these features have been implemented, a newly submitted character should appear abutting the right edge, and only move left once the follow character has been submitted.

LED Matrix – Incomplete Character Tier A: 6 marks

*Requires the **LED Matrix – Single Character** feature to be implemented.*

When a mark is inputted, the character that would correspond to the sequence up to that point should display on the LED matrix, in red. When the character is submitted, the glyph should turn green.

For example:

Input LED Matrix

"▬"

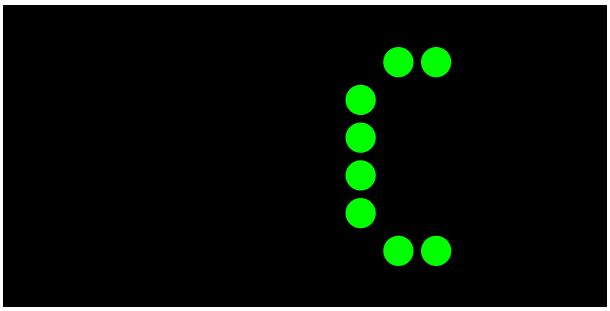
“●”

“▬”

“●”

THEN EITHER

“Ω”
(if **LED Matrix – Multiple Characters** has been implemented)



OR

“Ω”
(if <u>not</u>)

If **LED Matrix – Multiple Characters** is implemented, then the previously submitted characters should move left when it is submitted. This will be modified by the **Animation – LED Matrix** feature below.

Your implementation should make efficient use of the available LED matrix commands, and avoid sending gratuitously more commands than the minimum necessary. You will be asked to justify during your demo that the number of commands sent is appropriate.

Animation – IO Board — Tier A: 6 marks

*Requires the **Button Inputs** feature to be implemented.*

When a button is pressed, the LEDs on the IO board should animate the input being added. The moment the button is pressed, and then every 100ms after that, the LED pattern should move to the left, and the new LED added to the right edge, until the final LED has been added. For example:

Input	LEDs
(Initial)	
B0 – Dot	
(Hold)	
B1 – Dash	
(Tick)	
(Tick)	
(Tick)	
(Hold)	
B0 – Dot	
(Tick)	
(Hold)	
B1 – Dash	
(Tick)	
(Tick)	
(Tick)	
(Hold)	
B2 – Submit	
(Tick)	
(Tick)	
(Hold)	

The animation should not cause the LEDs to flicker, or display other such visual issues.

If an input is made while the previous animation is running, this should be handled in a reasonable manner.

Seven Segment Display — Tier A: 6 marks

*Requires the **LED Matrix – Multiple Characters** and **LED Matrix – Incomplete Character** features to be implemented.*

On the right digit of the seven segment display, show how many marks have been inputted for the in-progress character, if this number is non-zero. If it is zero, turn off this digit. If this number would go above nine, display "-" on the right digit.

On the left digit of the seven segment display, show how many complete characters have been submitted, in hexadecimal modulo 16, with the decimal point turned on if and only if the right digit is on.

If you have implemented **Animation – IO Board** and/or **Animation – LED Matrix** (below), you may choose to have the seven segment display update before the animation commences or after (either or both) animation(s) complete.

The seven segment display should show no visual issues, such as flickering or ghosting.

Serial Output — Tier A: 4 marks

*Requires the **LED Matrix – Incomplete Character** feature to be implemented.*

Output the characters to the serial terminal as they are being entered into the emulator. Submitted characters should be fixed in place, with new characters being added immediately (i.e. without a space) to the right (unlike the LED matrix, where the position of a new character is fixed, and older characters move left). Additionally, incomplete characters should be displayed as marks are entered, with each new mark causing the old incomplete character to be replaced by the updated character. Submitting twice in a row should print a space to the serial output (and a third submission onward should output nothing). For example (with underlining showing the current location of the cursor – this underlining should not be displayed):

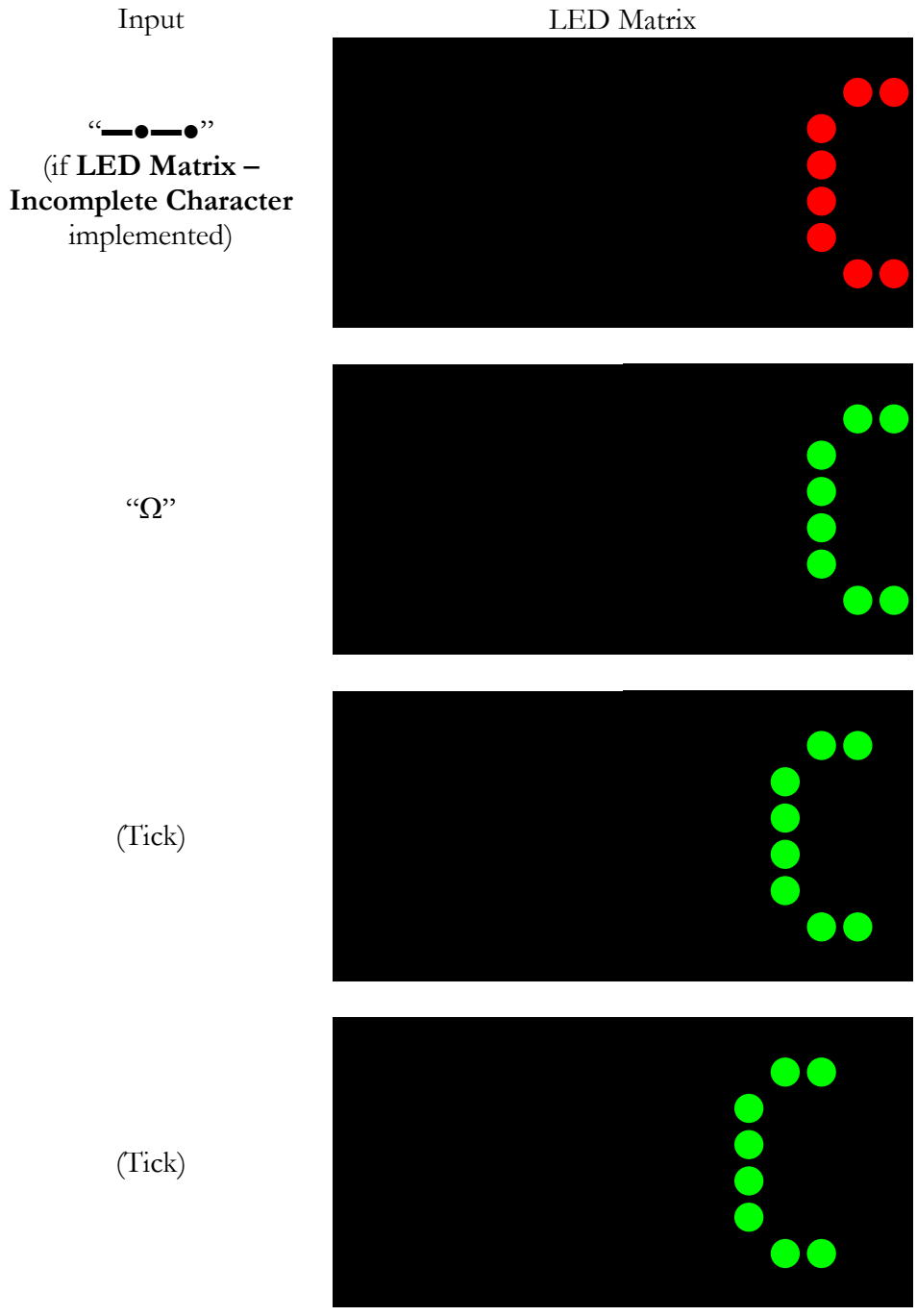
Input (Initial)	Terminal
"▬"	_
"●"	T_
"▬"	N_
"●"	K_
"Ω"	C_
"●"	C_
"●"	CE_
"●"	CI_
"Ω"	CS_
"Ω"	CS_

The serial terminal should show no visual issues, such as flickering. You must be able to display at least 200 characters on a terminal that is 80 columns wide and 24 rows tall. All outputted alphabetical characters should be capitalised (even if they were entered as lowercase with the **Serial Input** feature below).

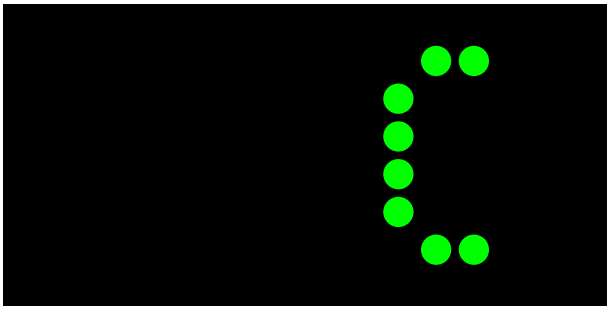
Animation – LED Matrix — Tier B: 8 marks

*Requires the **LED Matrix – Multiple Characters** feature to be implemented.*

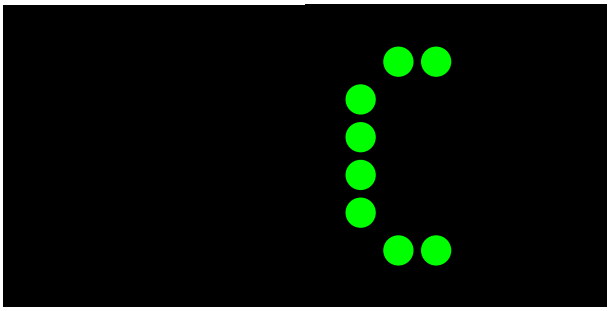
When a character is submitted, all glyphs should animate moving to the left. Every 100ms after the button is pressed, all pixels should move one column to the left, with the new column of the glyph added to the right edge, and continue until there is room for a new character to the right of the already submitted characters (i.e. four columns with the small font). The glyph should turn green or appear in green (depending on if **LED Matrix – Incomplete Character** has been implemented), but remain in place when the submit button is pressed, and only move left for the first time 100ms after that.



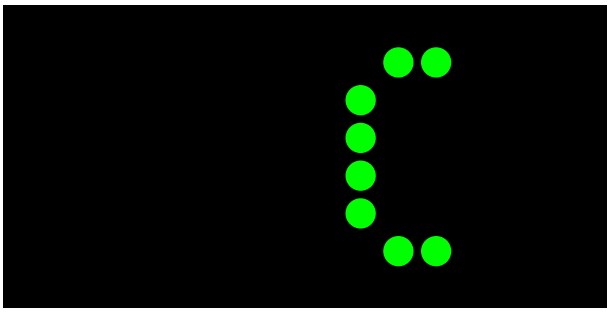
(Tick)



(Tick)



(Hold)



The animation should not cause the LED matrix to flicker, or display other such visual issues.

If an input is made while the previous animation is running, this should be handled in a reasonable manner.

Your implementation should make efficient use of the available LED matrix commands, and avoid sending gratuitously more commands than the minimum necessary. You will be asked to justify during your demo that the number of commands sent is appropriate.

Synchronous Mode — Tier B: 12 marks

*Requires the **LED Matrix – Incomplete Character**, **Animation – IO Board**, **Animation – LED Matrix** and **Seven Segment Display** features to be implemented.*

When switch S0 is low, the emulator should run in asynchronous mode, as described in **Button Inputs**. When switch S0 is high, the emulator should run in synchronous mode, which uses only button B0, and determines the inputs based on the duration of button presses and releases:

- If B0 is held down for less than 200ms, then a dot should be inputted;
- If B0 is held down for more than 200ms, then a dash should be inputted;
- If B0 is released for more than 1000ms, then a submit should be inputted;
- If B0 is released for more than 2000ms (1000ms after the previous submit), then a second submit should be inputted, resulting in a space;

- If two consecutive submits have been inputted, additional time spent with B0 released should have no effect (i.e. there should never be three consecutive submissions).

Pressing B1 or B2 (or B3) in synchronous mode should have no effect on the emulator.

All other features must work in both asynchronous mode and in synchronous mode.

Serial Input — Tier B: 6 marks

*Requires the **Animation – LED Matrix** feature to be implemented.*

When a character is typed into the serial terminal, if it has a valid Morse code pattern, that character should be echoed to the serial terminal, its glyph should be displayed on the LED matrix in yellow (and should animate moving left from the right edge), and its pattern should display on the IO board LEDs. If a character does not have a valid Morse code pattern, do nothing.

If you have implemented **LED Matrix – Incomplete Character**, and there is a character in progress, you should handle this in a reasonable manner.

If **Animation – IO Board** and/or **Animation – LED** Matrix has been implemented, the character's pattern should animate on the IO board LEDs, and/or the character's glyph should animate on the LED matrix.

Buzzer — Tier B: 8 marks

*Requires the **Animation – IO Board** and **Serial Input** feature to be implemented.*

When a valid character is inputted on the serial terminal, play its pattern on the buzzer, aligned with the animation on the IO board LEDs (dashes should play for longer than dots), such that, for the duration of the IO board animation, a tone should be playing while L0 is lit, and the buzzer should be silent while L0 is dark. Dashes and dots should have different frequencies, with one 1.25× to 1.5× higher than the other. Most marks should play at a 50% duty cycle, but the final mark in each pattern should be played at a 10% duty cycle (and as such, "E" and "T" will consist of solely a tone played at a 10% duty cycle).

During the gaps between marks, and after the final mark of a character, the buzzer should be silent.

For example:

Input	IO Board LEDs	Tone
(Initial)	⚫⚫⚫⚫⚫⚫⚫⚫	Silence
"C"	⚫⚫⚫⚫⚫⚫⚫🔵	Dot tone
(Tick)	⚫⚫⚫⚫⚫⚫🔵⚫	Silence
(Tick)	⚫⚫⚫⚫⚫🔵⚫🟢	Dash tone
(Tick)	⚫⚫⚫⚫🔵⚫🟢🟢	Dash tone continued
(Tick)	⚫⚫⚫🔵⚫🟢🟢🟢	Dash tone continued
(Tick)	⚫⚫🔵⚫🟢🟢🟢⚫	Silence
(Tick)	⚫🔵⚫🟢🟢🟢⚫🔴	Dot tone

Input	IO Board LEDs	Tone
(Tick)		Silence
(Tick)		Final dash tone
(Tick)		Final dash tone continued
(Tick)		Final dash tone continued
(Hold)		Silence

If an additional character is inputted into the serial terminal while the previous character's pattern is still being played on the buzzer, you should handle this in a reasonable manner.

Serial Colours — Tier C: 6 marks

*Requires the **LED Matrix – Incomplete Character**, **Serial Input** and **Serial Output** features to be implemented.*

When a character is displayed on the serial terminal, match the font colour to that of the LED matrix i.e.:

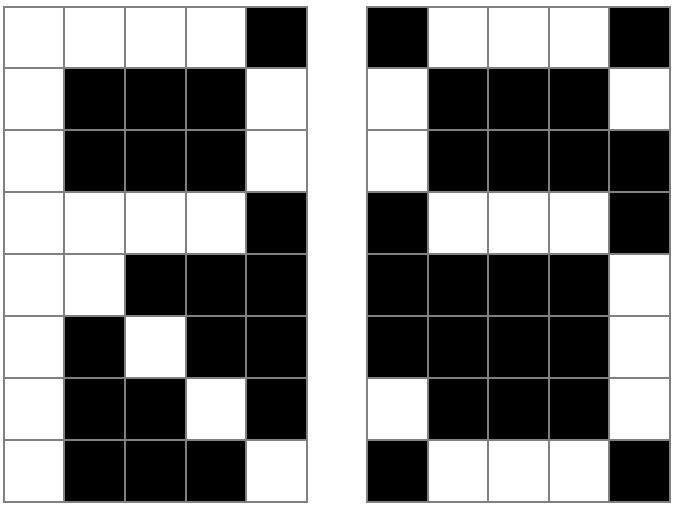
- An incomplete character should be displayed in red;
- A submitted character should be displayed in green; and
- A character inputted into the serial terminal should be displayed in yellow.

Font Selection — Tier C: 2 marks

*Requires the **LED Matrix – Multiple Characters** features to be implemented.*

Switch S1 controls the font selection. When S1 is low, the three-column font should be used. When S1 is high, the five-column font should be used. When S1 is switched, the LED matrix should update immediately.

For example, "R" and "S" are represented in the five-column font as `{ 0b11111111, 0b10011000, 0b10010100, 0b10010010, 0b01100001 }` and `{ 0b01100010, 0b10010001, 0b10010001, 0b10010001, 0b01001110 }`, respectively. This would produce the glyphs:



Your implementation should make efficient use of the available LED matrix commands, and avoid sending gratuitously more commands than the minimum necessary. You will be asked to justify during your demo that the number of commands sent is appropriate.

Joystick – Scroll — Tier C: 6 marks

*Requires the **Animation – LED Matrix** and **Font Selection** features to be implemented.*

When the joystick is tilted left and right (along the x axis), the LED matrix should scroll left and right, allowing glyphs that had previously slid off the screen to be seen once more. The scrolling should be done column-by-column (and not character-by-character) i.e. allow for a character to be partially visible along the edge of the LED matrix.

The angle of tilt should control the speed of the scroll e.g. a slight tilt right should scroll slowly into the past, while a complete tilt left should scroll quickly (but still coarsely controllable) to the present. This should be based on the current tilt of the joystick. As an example, if the joystick is tilted at a fast human speed from neutral to full tilt, it should immediately start scrolling quickly. It should not use the slight tilt that it detected early in the joystick's movement to initially scroll slowly, and only start scrolling quickly after the first cycle.

The glyphs for at least 50 previously submitted characters should be viewable. If the newest character is on the right edge of the screen, it should be impossible to scroll further left. A similar restriction applies to the oldest character when scrolling right.

If an input is made while viewing the scrollback, you should handle this in a reasonable manner.

The joystick need not interact with the IO board LEDs nor the serial terminal.

Your implementation should make efficient use of the available LED matrix commands, and avoid sending gratuitously more commands than the minimum necessary. You will be asked to justify during your demo that the number of commands sent is appropriate.

Joystick – Brightness — Tier C: 4 marks

*Requires the **LED Matrix – Single Character** feature to be implemented.*

When the joystick is tilted up and down (along the y axis), the brightness of the LED matrix should lighten and darken. Initially the LED matrix should be at full brightness. The brightness should not overflow, and at the lowest brightness level, the colours should still be discernible.

There should be 15 levels of brightness. The brightness should change by one level when tilted, and then every 1000ms, regardless of the angle of tilt (beyond any deadzone threshold). However, should the joystick be returned to the neutral position, this should nullify the timing, and so should immediately change the brightness if tilted again. As such, it should be possible to change the brightness at a rate faster than every 1000ms by rapidly moving the joystick between tilted and neutral. However, it should not be possible to hold the joystick at a borderline between tilted and neutral in such a way that natural fluctuations in the analogue signal cause the brightness to change at high speed.

Your implementation should make efficient use of the available LED matrix commands, and avoid sending gratuitously more commands than the minimum necessary. You will be asked to justify during your demo that the number of commands sent is appropriate.

EEPROM — Tier C: 6 marks

*Requires the **Serial Input** and **Joystick – Scroll** features to be implemented.*

Remember the scrollback (including colour) between power cycles using the EEPROM. Only submitted characters should be stored, and you should be able to store at least 50 characters. You should structure your storage method to distribute which EEPROM locations you write to as much as practical. Any location should not be written to more than once every 50 inputs on average (to clarify, this is averaged over the inputs, but must be true of all locations – it is not averaged over the locations). You will need to justify your distribution method during your demo.

Assessment of Feature Implementation

The program improvements will be worth the number of marks shown above. You will be awarded marks for each feature up to the maximum mark for that feature. Part marks will be awarded for a feature if only some part of the feature has been implemented or if there are bugs/problems with your implementation (which may include issues such as incorrect data direction registers). Your additions to the program must not negatively impact its usability, visual appearance, or user experience. Bad C, AVR or general coding practises that can be observed from the functionality of your implementation (such as spamming the terminal) may also lead to deductions, even if not explicitly stated in the feature descriptions. Note also that the features you implement must appropriately work together, for example, if you play sound, then any other features must still work while the sound is playing. Furthermore, features must work for all relevant inputs, and must continue to work for any number of relevant inputs. In addition, features that ask for two things to be different must be different enough that the differences are obvious without close inspection. If they are not sufficiently different, marks might not be awarded. For example, if a feature asked for two different audio tones to be produced, and one tone was, say, 440Hz and the other was 441Hz, this would be insufficiently different. In general, any implementation that correctly implements features according to a reasonable interpretation of this document will be awarded full marks. However, an implementation that fails to show academic merit with respect to the content a given feature was designed to assess will be deemed in unreasonable interpretation of that feature.

Your code must compile and function under laboratory conditions, specifically the conditions of the Axon computer lab. The code will be compiled on Microchip Studio on a lab computer as demonstrated in the lab sessions throughout the semester. You must attend your allocated demo session with your CSSE2010 kit wired up as needed. Students should test their submission on the lab computer. The code will be compiled as-is, and students cannot request alterations to the code to enable or disable various features.

Submission Details

The due date for the AVR assignment is 4:00pm Friday 22nd May. The AVR assignment must be submitted via BlackBoard. You must electronically submit a single `.zip` file containing all of the C source files (`.c` and `.h`) necessary to build the project (including any that were provided to you – even if you have not changed them).

Do not submit `.rar` or other archive formats – the single file you submit must be a `.zip` format file. All files must be at the top level within the `.zip` file – do not use folders/directories or other `.zip`/`.rar` files inside the `.zip` file.

If you make more than one submission, each submission must be complete – the single `.zip` file must contain all source files needed to build your work. Only your last submission will be marked, and the submission time of this submission will be used to calculate any late penalties.

It is the responsibility of the student to ensure that their submission is correctly uploaded to the BlackBoard submission portal with all of the files they intend to submit. This can be ensured by downloading your `.zip` file after submission is made, un-zipping the submission to check all files are correctly included and checking whether you are able to compile the project successfully. It is suggested that you use the Axon lab computers for this check.

After the due date, you code will undergo some automated testing. If this testing flags any issues, you will be emailed by a course staff member, advising you of what issues were flagged. Historically, this process has had an 80+% positive predictive value, but you should still verify the issue if you are emailed. Should you choose to resubmit, late penalties will be applied. Should you choose to not resubmit, your code will be marked as-is, and any problems that may arise from the flagged issues will be penalised appropriately. This process has also historically had a 95+% negative predictive value, but nonetheless, a failure to receive an email does not guarantee that your submission is error free, and penalties may still be applied.

"Use of AI" Declaration

The BlackBoard submission link will contain a text field for declaring any AI usage you undertook in the course of this assignment. This section is mandatory, though may be answered with "no AI usage" if appropriate. You should keep notes on your AI usage while you complete this assignment to ensure that you give an accurate answer to this question.

Incomplete or Invalid Code

If your submission is missing files (e.g., will not compile and/or link due to missing files) then we will substitute the original files as provided to you. No penalty will apply for this, but no changes you made to the missing files will be considered in marking.

If your submission does not compile and/or link in Microchip Studio as installed on the lab computers for other reasons, then the marker will make reasonable attempts (without constructively adding to your code) to get your code to compile and link by fixing a small number of simple syntax errors and/or commenting out code which does not compile. For clarification, the goal of the marker during this fixing process is to get the code to compile, and not necessarily to get the code to function, and they are likely to use brute force to accomplish this goal. A penalty of between 10% and 50% of your mark will apply depending on the number of corrections required. This will apply based off of compiling and linking with Microchip Studio on the Axon lab computers, regardless of the results of compiling or linking with other programs. If the marker is unable to get your submission to compile and/or link by these methods, then you will receive 0 for the AVR assignment. A minimum 10% penalty will apply, even if only one character needs to be fixed.

Compilation Warnings

If there are compilation warnings when building your code, then a mark deduction will apply – 1 mark penalty per warning for the first ten warnings. The warning list generated by Microchip Studio on the Axon lab computers from a clean build will be the canonical list for these penalties. To check for warnings, rebuild all your source code (choose "Rebuild Solution" from the "Build" menu in Microchip Studio) and check for warnings in the "Error List" tab. If you are using other software (such as PlatformIO) for your assignment, you are encouraged to perform a final build on Microchip Studio on the lab computers.

General Deductions

If there are problems in your submitted AVR project that do not fit into any of the above feature categories, then some marks may be deducted for these problems. This could include problems such as general lag, errors introduced to the original program etc. The number of marks deducted will depend on the severity of the issues.

The use of blocking delays in your code will result in a deduction, regardless of if this can be detected in the functionality of your code.

Late Submissions

As per the course profile, where an assessment item is submitted after the deadline (i.e., 4:00pm Friday 22nd May), without an approved extension, a late penalty will apply. Assessment submissions received after the due time (or any approved extended deadline) will be subject to a 10% late penalty per day (or part thereof) of the maximum possible assignment mark (i.e. equivalent to 10 feature marks), up to a maximum of seven days. After seven days, a 100% late penalty will be applied.

Requests for extensions should be made via the process described in the course profile (before the due date) and be accompanied by documentary evidence of extenuating circumstances (e.g., medical certificate). The application of any late penalty will be based on your latest submission time.

Notification of Results

Students will be notified of their results via the My Grades section of BlackBoard.

CSSE7201

The AVR programming described in this document constitutes part two of the CSSE7201 assessment. Please see BlackBoard for part one, and/or the course profile for more information regarding the assessment details.

Version

This is version three of the assignment document. Changes between the current version and the previous version are shown in green. Changes between the previous version and version one are shown in blue.